CAR SERVICE BACKEND

Learning Guide for the Project Stack

A practical roadmap of the libraries, infrastructure, and backend topics needed to understand, maintain, and extend this NestJS car service platform.

Backend Framework

NestJS 11 with TypeScript, modules, controllers, services, guards, and DTO validation.

Data Layer

PostgreSQL, Prisma 7, migrations, generated client, and relational schema design.

Async Work

Redis, BullMQ, notification outbox, background jobs, retries, and fallback behavior.

Contents

Generated for this repository on June 26, 2026.

Project Stack At A Glance

This backend is a modular NestJS API. The main app imports modules for auth, account, onboarding, bookings, dashboard, services, business, customers, vehicles, inventory, payments, reports, billing, inbox, support, locations, mobile, notifications, cache, queues, and Prisma.

NestJS TypeScript PostgreSQL Prisma Redis BullMQ JWT FCM Docker Compose

AREA	WHAT TO LEARN	WHY IT MATTERS HERE
Backend architecture	Nest modules, dependency injection, controllers, services, providers, guards.	Every feature is split into modules such as bookings, mobile, notifications, and account.
Persistence	Prisma schema, migrations, generated client, transactions, relational modeling.	Users, businesses, bookings, inbox messages, device tokens, payments, and inventory live in PostgreSQL.
Notifications	FCM tokens, service accounts, push payloads, in-app inbox, notification preferences.	Booking creation, status changes, and cancellations create inbox rows and optionally push to devices.
Queues and cache	Redis, BullMQ queues, retries, backoff, idempotency, cache invalidation.	The backend already has queue/cache infrastructure and Redis in Docker Compose.
Security	JWT, refresh tokens, Passport strategies, role guards, argon2 password hashing.	The app has separate business, client, and admin roles with protected routes.

High value mental model: a request enters a controller, validation shapes the DTO, a service runs business logic, Prisma persists state, cache/queues handle side effects, and guards keep users inside the routes they are allowed to use.

Core Backend Libraries

LIBRARY	LEARN THESE TOPICS	PROJECT USE
`@nestjs/common` `@nestjs/core`	Modules, providers, decorators, exceptions, pipes, guards, interceptors.	The foundation of every API feature and dependency injection path.
`@nestjs/config` `dotenv`	Environment variables, config loading, feature flags, secret handling.	Used for database, Redis, JWT, queue flags, and Firebase credentials.
`@prisma/client` `prisma`	Schema models, migrations, relations, generated types, query filters.	Main ORM for bookings, users, businesses, inbox, notification devices, and more.
`@prisma/adapter-pg` `pg`	PostgreSQL connections, pools, adapters, connection strings.	Prisma connects to PostgreSQL through the PG adapter and a connection pool.
`class-validator` `class-transformer`	DTO validation, type conversion, optional fields, enums, arrays.	Global validation pipe validates request bodies and query params.
`@nestjs/jwt` `passport-jwt`	Access tokens, refresh tokens, JWT claims, auth guards.	Protects most routes and carries user id, email, role, and business id.
`argon2`	Password hashing, verification, salts, secure login design.	Business and client passwords are hashed before storage.
`rxjs` `reflect-metadata`	Nest internals, decorators, metadata, observables basics.	Required by NestJS and useful when building more advanced providers.

Developer Tooling

TOOL	LEARN THESE TOPICS	PROJECT USE
`typescript`	Types, unions, generics, strict null checks, module resolution.	All backend code is TypeScript targeting modern Node.

TOOL	LEARN THESE TOPICS	PROJECT USE

`jest`
`ts-jest`
`supertest`
	Unit tests, integration tests, HTTP e2e tests, mocks.	The project has test wiring and is ready for focused backend specs.

`eslint` `prettier`	Lint rules, auto formatting, consistent code style.	Used by the lint and format scripts.
`Docker Compose`	Local services, volumes, health checks, ports.	Runs PostgreSQL 16 and Redis 7 for local development.

Notifications, FCM, Redis, And BullMQ

Firebase Cloud Messaging

- Learn FCM registration tokens and how each device gets one.
- Learn service account credentials and Firebase HTTP v1 API.
- Learn notification vs data payloads and deep links.
- Learn token cleanup for expired or unregistered devices.
- Learn web push service workers and VAPID keys for browsers.

Project Notification Flow

- Register token with `POST /notifications/devices`.
- Store token in `NotificationDevice`.
- Create `InboxMessage` for every important event.
- Send push when Firebase credentials are configured.
- Disable invalid tokens after failed FCM delivery.

NEED	BACKEND LIBRARY OR TOPIC	MOBILE OR WEBSITE COMPANION
Send push notifications	Firebase HTTP v1, or optional **`firebase-admin`**.	Firebase JS SDK for web, Firebase Messaging for Android/iOS.
Device registration	DTO validation, Prisma upsert, unique FCM token storage.	Request permission, get token, refresh token when FCM rotates it.
In-app inbox	**`InboxMessage`**, unread/read state, JSON data payload.	Notifications tab, badge count, deep link to booking details.
Background jobs	**`@nestjs/bullmq`**, **`bullmq`**, queue processors.	No direct client library needed. UI should show eventual status when jobs finish.
Redis connection	**`ioredis`**, Redis URL parsing, retry behavior.	No direct client library needed. It supports backend cache and queues.

BullMQ Topics To Learn

- □ Queues, jobs, workers, processors, and job names.
- □ Attempts, exponential backoff, delayed retries, and failed jobs.
- □ Idempotency: avoid sending the same notification twice.
- □ Outbox pattern: save the side effect before or during delivery.
- □ Operational monitoring: Redis memory, stalled jobs, and queue dashboards.

Redis Topics To Learn

- □ Key/value basics, TTL, eviction, persistence, and connection strings.
- □ Redis as a queue backend for BullMQ.
- □ Redis as a cache backend for dashboard and setup warning read models.
- □ Local Redis through Docker Compose and production Redis hosting.

Database, Auth, API, And Deployment Topics

Prisma And PostgreSQL

- Model relations: user to business, customer to car, booking to service.
- Migrations and when to use `migrate dev` vs `migrate deploy`.
- Indexes for common filters such as booking status, dates, and unread inbox.
- Transactions for multi-step writes.
- JSON fields for preferences and flexible mobile state.

Authentication And Authorization

- JWT access token structure and expiration.
- Refresh token rotation and revocation.
- Role guards for client, business, and admin users.
- Password hashing with argon2.
- MFA fields and security preferences.

REST API Design

- Controllers, route naming, and HTTP status codes.
- DTOs for request bodies and query params.
- Pagination responses and filtering.
- OpenAPI/Swagger contracts and frontend coordination.
- Error handling through Nest exceptions.

Testing

- Unit tests for service methods and serializers.
- E2E tests with Supertest for auth, bookings, and notifications.
- Mocking Prisma and external FCM delivery.
- Testing queue fallback behavior when Redis is disabled.
- Seed data for predictable booking scenarios.

Deployment And Ops

- Environment variables and secret management.
- Database migrations in production startup.
- Redis availability and queue workers.
- Logging failed notification delivery.
- CORS configuration for website and mobile clients.

Useful Commands

- `npm run db:dev:up` starts PostgreSQL and Redis.
- `npm run prisma:migrate` creates and applies local migrations.
- `npm run prisma:generate` refreshes Prisma client types.
- `npm run build` verifies TypeScript compilation.
- `npm test` runs Jest specs when spec files exist.

Suggested Learning Order

PHASE	FOCUS	OUTCOME
1	TypeScript, Node.js, npm scripts, environment variables.	You can run the project, read types, and understand build errors.
2	NestJS modules, controllers, services, guards, DTO validation.	You can follow a request from route to database and back.
3	Prisma, PostgreSQL, migrations, schema relations.	You can safely add fields, tables, indexes, and queries.
4	Auth: JWT, Passport, refresh tokens, roles, argon2.	You can protect routes and reason about user sessions.
5	Bookings domain: customers, cars, services, status changes.	You can change business workflows without breaking API responses.
6	Notifications: inbox, preferences, FCM, device tokens.	You can send booking updates to website and mobile users.
7	Redis and BullMQ: queues, retries, cache, background processing.	You can move slow work out of HTTP requests and operate queues.
8	Testing, OpenAPI, deployment, monitoring.	You can ship changes with confidence and debug production issues.

Minimum Practical Checklist

- □ Run the backend locally with PostgreSQL and Redis.
- □ Create a business account, client account, vehicle, service, and booking.
- □ Trace the booking response shape from controller to serializer.
- □ Add a small Prisma field and create a migration.
- □ Register a notification device token and inspect the database row.
- □ Change a booking status and verify inbox notification creation.
- □ Turn Redis queues on and observe queue behavior.

- [ ] Add one Jest test for a booking or notification service method.

Recommended first deep dive: learn NestJS and Prisma together. Those two explain most of the code. After that, Redis/BullMQ and FCM will feel like extensions of the same service architecture instead of separate mysteries.